Khmelnytskyi National University
Department of Information Technonogy

Report

Laboratory work №5

Discipline: “Programming”

Topic: “*Two-dimensional arrays*”

Completed: 1st year student, group SE-26-1 Nykon Hrytsyshyn
Name, Surname

Checked: Valeriy Martynyuk
Name, Surname

Khmelnytskyi, 2026

Purpose: learn to work with two-dimensional arrays and program them in the C programming language.

Technical Environment: The project is implemented in accordance with the C11 standard and configured for cross-platform compilation using CMake and the Ninja build system. The application is built and verified using modern C compilers (GCC, Clang, and MSVC) with strict diagnostic flags enabled, ensuring high code quality, portability, and compatibility across operating systems.

Methodology & Implementation Overview: Two-dimensional arrays (matrices) in C are represented in contiguous memory using row-major order, where element addresses are calculated via two-dimensional indexing A[i][j]. To handle arbitrary matrix dimensions up to a predefined upper bound (MAX_MATRIX_DIM = 15), a fixed-size contiguous buffer is allocated, while logical matrix boundaries (rows and cols) are passed dynamically to helper functions.

The software implementation utilizes a unified matrix processing module (matrix.h / matrix.c) providing standard primitives for size validation, manual input, pseudo-random generation with custom range bounds via rand(), and formatted console visualization. Robust input streams ensure user inputs remain within valid ranges.

Note: The software implementation is designed using a modular architecture with robust input validation and an interactive task execution engine. Detailed flowcharts illustrating the overall execution logic, along with full source code for all program components (tasks.h, matrix.h, utils.h, main.c, runner.c, matrix.c and utils.c), are provided in Appendices A…H.

Task 1

Given matrix A of dimension 6×3. Select an element A[i, j] arbitrarily and delete the column in which it is located.

1.1. Block diagram of the program algorithm

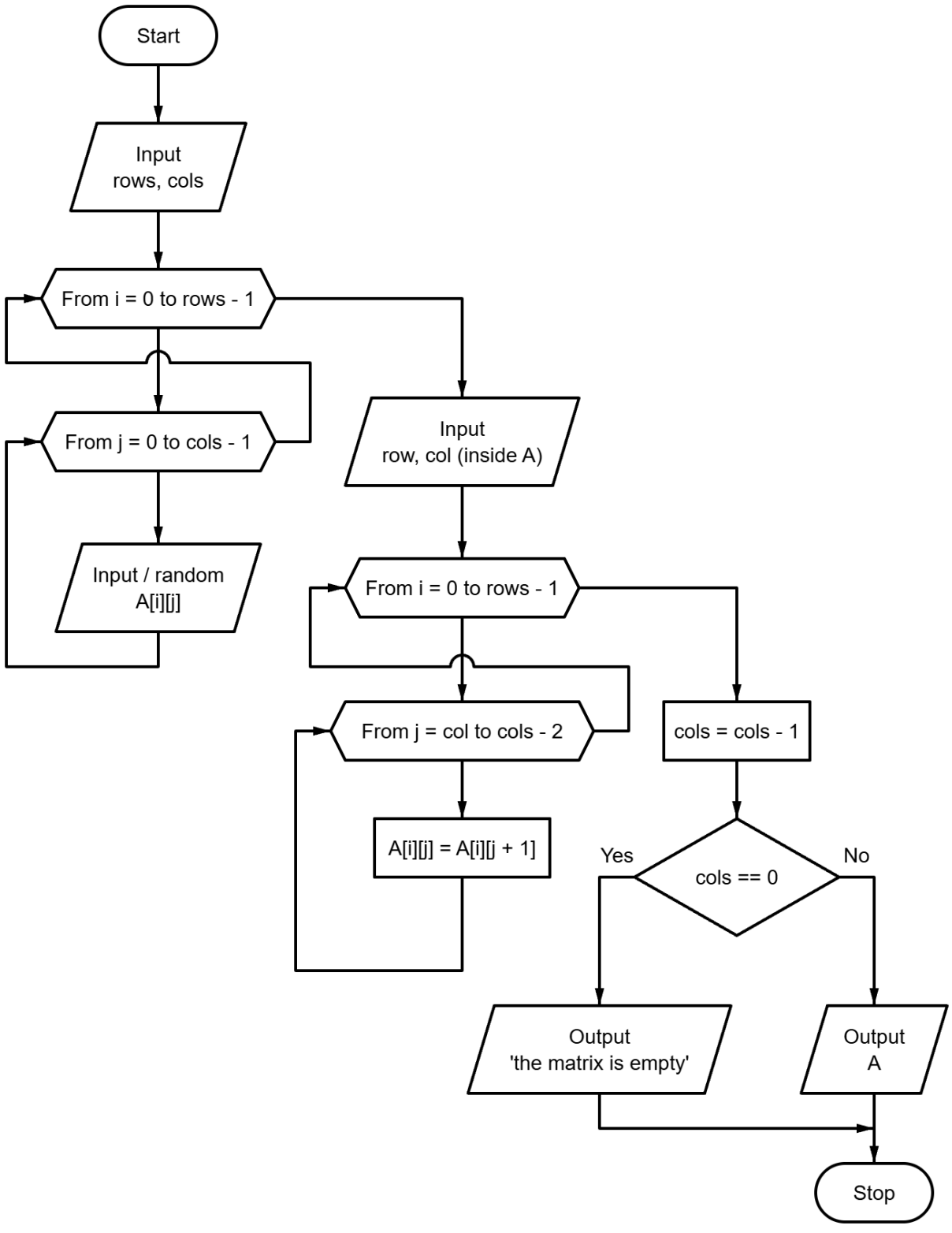


Fig. 1 – Block diagram of the program algorithm

1.2. Program text

```
#include "matrix.h"
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

void run_task1(void) {
    Matrix a;
    long rows = 0;
    long cols = 0;
    long row_id = 0;
    long col_id = 0;

    puts("\n=== Task 1: delete the column of a chosen element
===\n");

    if (!read_matrix_size("A", &rows, &cols) ||
        !read_matrix("A", a, rows, cols)) {
        return;
    }

    puts("");
    print_matrix("A", a, rows, cols);
    puts("");

    if (!read_long_in_range(
            "Enter the row i of the element A[i][j]: ", 0,
rows - 1, &row_id
        ) ||
        !read_long_in_range(
            "Enter the column j of the element A[i][j]: ", 0,
cols - 1, &col_id
        )) {
        return;
    }

    printf(
        "\nChosen element: A[%ld][%ld] = %ld\n",
        row_id,
        col_id,
        a[row_id][col_id]
    );

    for (long i = 0; i < rows; ++i) {
        for (long j = col_id; j < cols - 1; ++j) {
            a[i][j] = a[i][j + 1];
```

```
        }
    }

    --cols;

    printf("Column %ld is deleted.\n\n", col_id);

    if (cols == 0) {
        puts("No columns are left, the matrix is empty.");
        return;
    }

    print_matrix("A", a, rows, cols);
}
```

1.3. Program results

```
=== Task 1: delete the column of a chosen element ===

Enter the number of rows of A: 6
Enter the number of columns of A: 2
Fill A with random numbers? (Y/n):

A (6 x 2) =
-10   1
 35  40
 32 -24
-39  49
-50  31
-43  37

Enter the row i of the element A[i][j]: 4
Enter the column j of the element A[i][j]: 1

Chosen element: A[4][1] = 31
Column 1 is deleted.

A (6 x 1) =
-10
 35
 32
-39
-50
-43
```

Fig. 2 – Task 1 results

Task 2

Given matrix A of dimension 7×9. Find the arithmetic mean of its elements and the number of elements smaller than the arithmetic mean.

2.1. Block diagram of the program algorithm

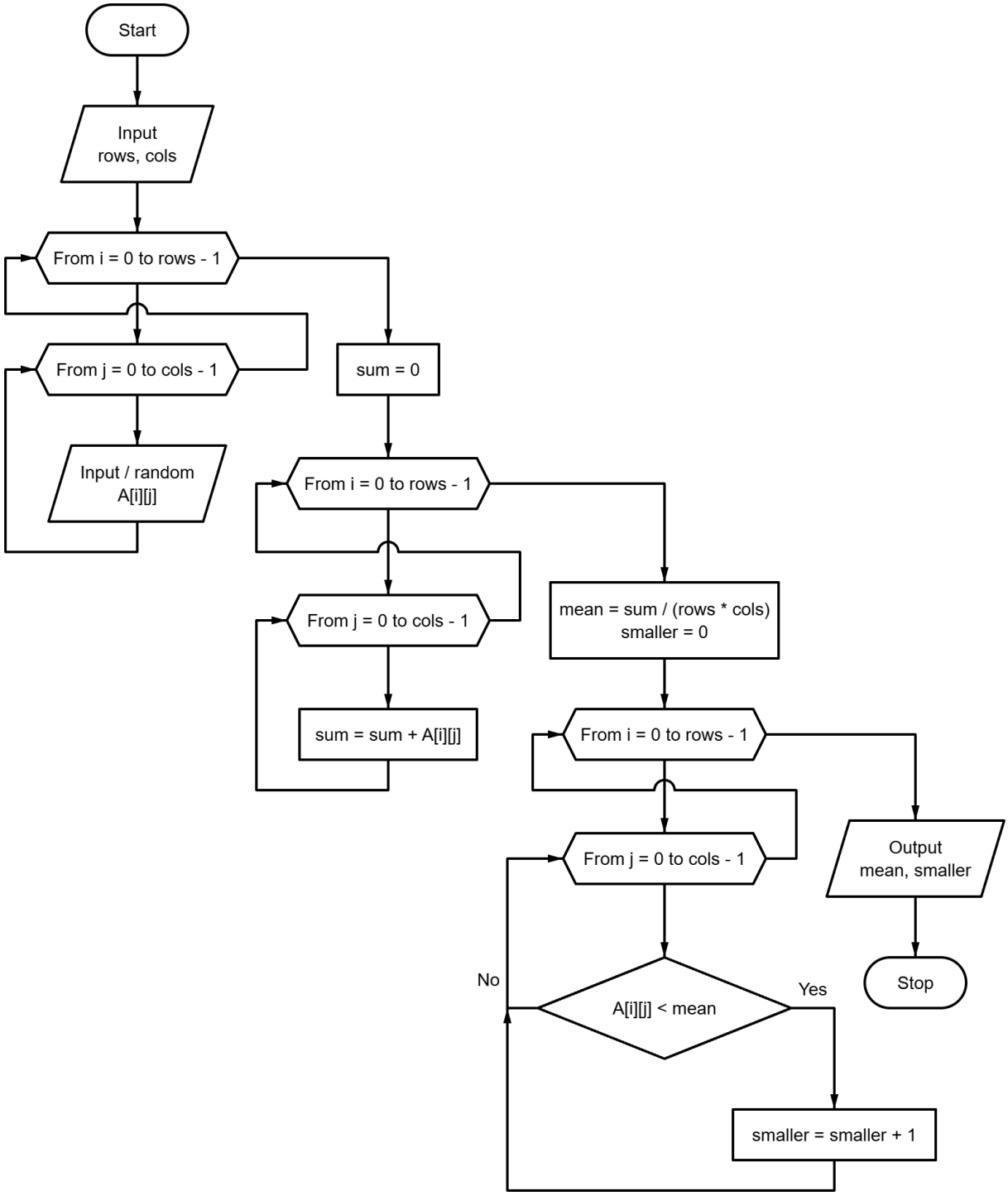


Fig. 3 – Block diagram of the program algorithm

2.2. Program text

```
#include "matrix.h"
#include "tasks.h"

#include <stdio.h>

void run_task2(void) {
    Matrix a;
    long rows = 0;
    long cols = 0;

    puts("\n=== Task 2: mean of the elements and the smaller ones
===\n");

    if (!read_matrix_size("A", &rows, &cols) ||
        !read_matrix("A", a, rows, cols)) {
        return;
    }

    long sum = 0;

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            sum += a[i][j];
        }
    }

    const double mean = (double)sum / (double)(rows * cols);
    int smaller_count = 0;

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            if ((double)a[i][j] < mean) {
                ++smaller_count;
            }
        }
    }

    puts("");
    print_matrix("A", a, rows, cols);
    printf("\nArithmetic mean: %.4f\n", mean);
    printf("Number of elements smaller than the mean: %d\n",
smaller_count);
}
```

2.3. Program results

```
=== Task 2: mean of the elements and the smaller ones ===

Enter the number of rows of A: 7
Enter the number of columns of A: 9
Fill A with random numbers? (Y/n):

A (7 x 9) =
-23  42 -22 -25  42  41  44  25   8
 -7 -17 -27 -43  46  35  25  47  26
-18  47  37  -3 -35  31  44 -13  21
-40  27 -20  38  40 -10  30  22 -21
 26  34 -31 -31  49 -36  16 -20   6
 13 -27 -17 -24  16 -22  -4  32  21
 40  21  21 -40  -3  23  41  40 -47

Arithmetic mean: 7.7937
Number of elements smaller than the mean: 28
```

Fig. 4 – Task 2 results

Task 3

Given matrix A of dimension 7×9 and matrix B of dimension 5×4. Perform A×B multiplication, record the result in the C matrix.

3.1. Block diagram of the program algorithm

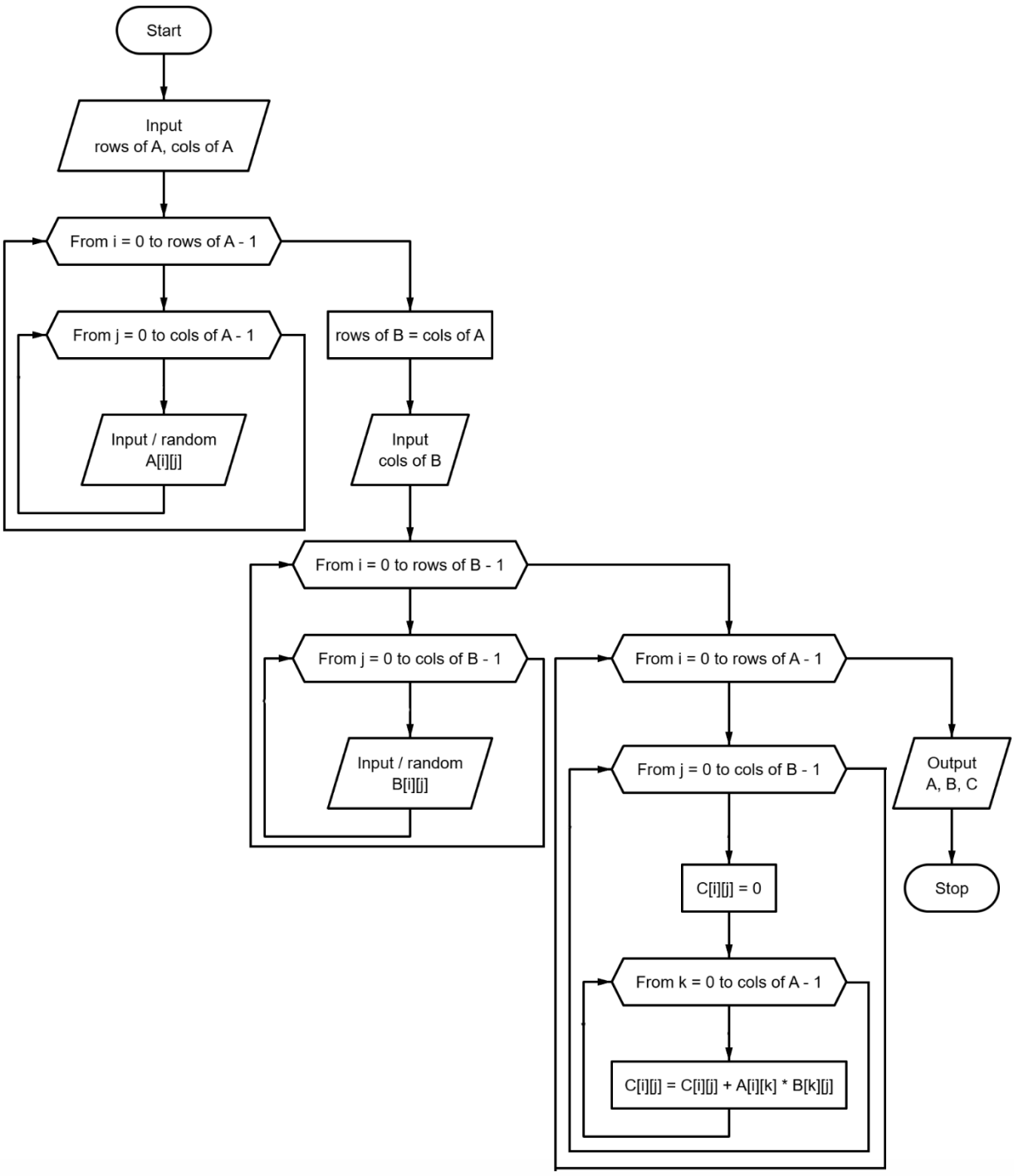


Fig. 5 – Block diagram of the program algorithm

3.2. Program text

```
#include "matrix.h"
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

void run_task3(void) {
    Matrix a;
    Matrix b;
    Matrix c;

    long a_rows = 0;
    long a_cols = 0;
    long b_cols = 0;

    puts("\n=== Task 3: product of matrices C = A x B ===\n");

    if (!read_matrix_size("A", &a_rows, &a_cols) ||
        !read_matrix("A", a, a_rows, a_cols)) {
        return;
    }

    const long b_rows = a_cols;

    printf("\nB must have %ld rows (the number of columns of
A).\n", b_rows);

    if (!read_long_in_range(
            "Enter the number of columns of B: ", 1,
MAX_MATRIX_DIM, &b_cols
        ) ||
        !read_matrix("B", b, b_rows, b_cols)) {
        return;
    }

    for (long i = 0; i < a_rows; ++i) {
        for (long j = 0; j < b_cols; ++j) {
            c[i][j] = 0;

            for (long k = 0; k < a_cols; ++k) {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }

    puts("");
```

```
    print_matrix("A", a, a_rows, a_cols);
    puts("");
    print_matrix("B", b, b_rows, b_cols);
    puts("");
    print_matrix("C", c, a_rows, b_cols);
}
```

3.3. Program results

```
=== Task 3: product of matrices C = A x B ===

Enter the number of rows of A: 7
Enter the number of columns of A: 9
Fill A with random numbers? (Y/n):

B must have 9 rows (the number of columns of A).
Enter the number of columns of B: 4
Fill B with random numbers? (Y/n):

A (7 x 9) =
 37 -11  17  11  19 -28  18  26  32
-13 -26  27 -48   0 -32  22 -17  25
 35 -50 -32 -10  18   4 -16 -28   1
  8 -41 -37   8  35  -4  50  -4   2
  3   5  14  37 -39  -3   0  34  24
-12  21 -24   2  -2  39  36 -38 -39
 41  27  24 -50  17 -38  32 -22  -9

B (9 x 4) =
-42 -11  11 -40
 11  50 -24 -10
-27  33  12   4
-39 -17   1  14
 14 -50  -4  28
-27  19  42  19
-30 -37  -1  -2
 -5  26  38 -28
 15 -23 -25 -25

C (7 x 4) =
-1731 -2791  -196 -2712
 2067 -1889 -1880  -585
   13 -4754   214   203
 -952 -7321    76   920
-2167  2275   840 -2297
-1251  -226   219  2854
  156  -580 -2266 -1983
```

Fig. 6 – Task 3 results

Conclusions

During the execution of this laboratory work, practical skills were acquired in processing two-dimensional arrays (matrices) in C11, implementing fundamental matrix manipulation algorithms, dynamically altering matrix dimensions, calculating statistical properties across rows and columns, and performing algebraic matrix operations.

APPENDIX A

Block diagram of the main program control flow algorithm

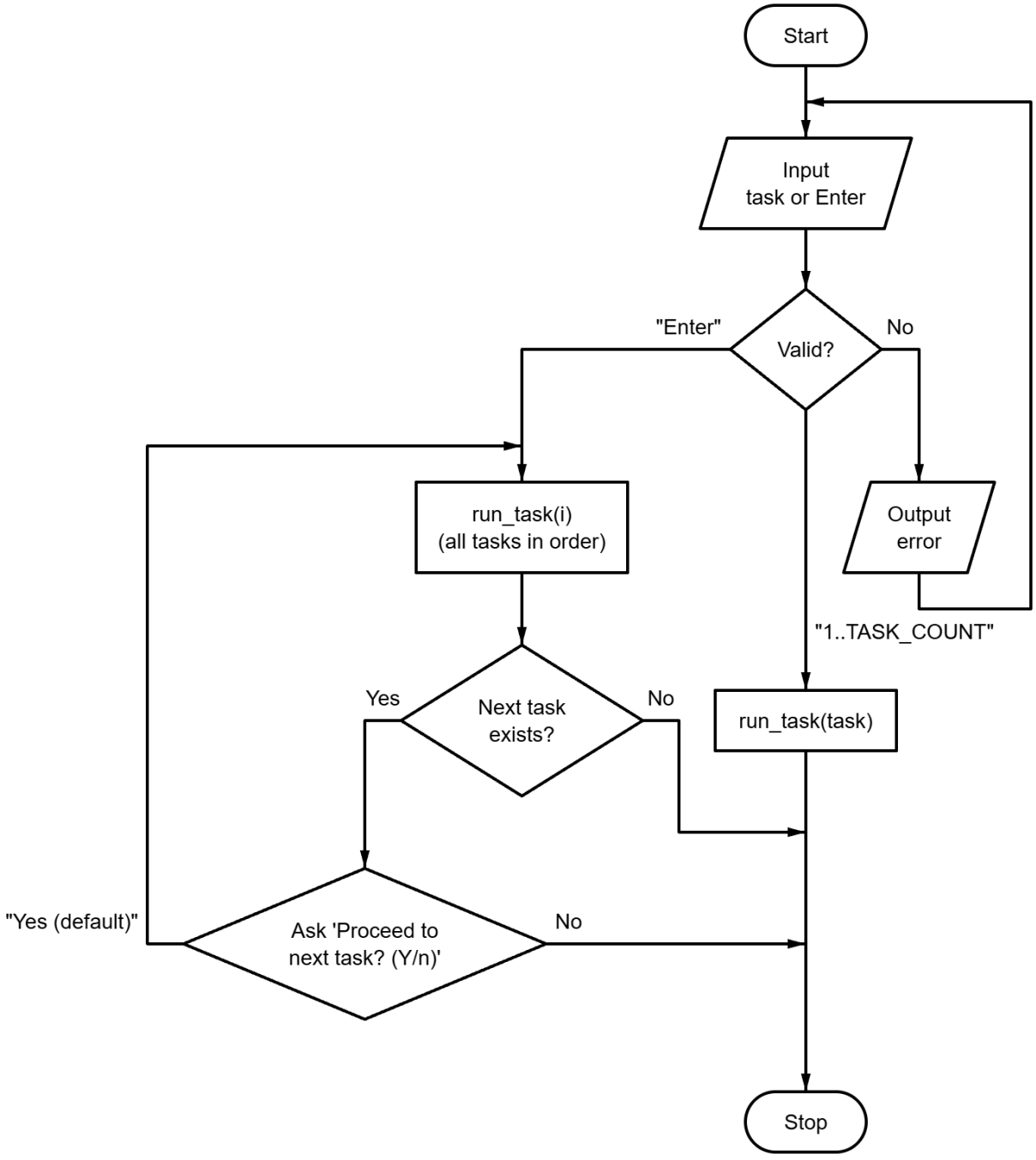


Fig. A.1 – Block diagram of the main program control flow algorithm

APPENDIX B

Source code of the “tasks.h” file

```
#ifndef TASKS_H
#define TASKS_H

#define TASK_LIST X(1) X(2) X(3)

#define X(num) void run_task##num(void);
TASK_LIST
#undef X

#define X(num) +1
enum { TASK_COUNT = 0 TASK_LIST };
#undef X

void run_task(int n);
void run_all_tasks(void);

#endif // TASKS_H
```

APPENDIX C

Source code of the “matrix.h” file

```
#ifndef MATRIX_H
#define MATRIX_H

#include <stdbool.h>

/* Max size of a prompt for user input */
#define PROMPT_SIZE 96

/* Max number of rows and columns of a matrix */
#define MAX_MATRIX_DIM 15

/* Max absolute value of a matrix element */
#define MAX_ELEMENT_ABS 1000L

/* Elements of a randomly filled matrix are from -RANDOM_ABS to
RANDOM_ABS */
#define RANDOM_ABS 50

typedef long Matrix[MAX_MATRIX_DIM][MAX_MATRIX_DIM];

bool read_matrix_size(const char *name, long *out_rows, long
*out_cols);
bool read_matrix(const char *name, Matrix matrix, long rows, long
cols);
void print_matrix(const char *name, Matrix matrix, long rows, long
cols);

#endif // MATRIX_H
```

APPENDIX D

Source code of the “utils.h” file

```
#ifndef UTILS_H
#define UTILS_H

#include <stdbool.h>
#include <stddef.h>

#define MAX_ARRAY_SIZE 100
#define MAX_ELEMENT_ABS 1000000L

bool read_double(const char *prompt, double *out_value);
bool read_doubles(const char *prompt, size_t count, ...);
bool read_long(const char *prompt, long *out_value);
bool read_longs(const char *prompt, size_t count, ...);

bool read_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value
);
bool read_optional_long_in_range(
    const char *prompt,
    long min,
    long max,
    long *out_value,
    bool *out_provided
);

bool read_sequence(const char *name, long *array, long
*out_count);
void print_sequence(const char *name, const long *array, long
count);

bool confirm(const char *prompt, bool default_answer);

#endif // UTILS_H
```

APPENDIX E

Source code of the “main.c” file

```
#include "tasks.h"
#include "utils.h"
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define PROMPT_SIZE 96

int main(void) {
    puts(
"========================================================\n"
        "  Laboratory work 5: Two-dimensional arrays (variant
9)\n"
"========================================================\n");

    srand((unsigned)time(NULL)); // NOLINT(*-msc51-cpp)

    char prompt[PROMPT_SIZE];
    snprintf(
        prompt, sizeof(prompt),
        "Enter task number (1-%d) or press Enter to run all tasks
[ALL]: ", TASK_COUNT
    );

    long task = 0;
    bool provided = false;

    if (!read_optional_long_in_range(prompt, 1, TASK_COUNT, &task,
&provided)) {
        return 1;
    }

    if (provided) {
        run_task((int)task);
    } else {
        run_all_tasks();
    }

    printf("\nPress Enter to exit...");
    getchar();

    return 0;
}
```

APPENDIX F

Source code of the “runner.c” file

```
#include "tasks.h"
#include "utils.h"

#include <stdio.h>

static void (*const TASKS[TASK_COUNT])(void) = {
#define X(num) run_task##num,
    TASK_LIST
#undef X
};

void run_task(const int n) {
    TASKS[n - 1]();
}

void run_all_tasks(void) {
    for (int i = 1; i <= TASK_COUNT; ++i) {
        run_task(i);

        if (i < TASK_COUNT) {
            char prompt[64];
            snprintf(
                prompt, sizeof(prompt), "\nProceed to task %d?
(Y/n): ", i + 1
            );

            if (!confirm(prompt, true)) {
                break;
            }
        }
    }
}
```

APPENDIX G

Source code of the “matrix.c” file

```
#include "matrix.h"

#include "utils.h"

#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

bool read_matrix_size(
    const char *const name,
    long *const out_rows,
    long *const out_cols
) {
    char prompt[PROMPT_SIZE];

    snprintf(prompt, sizeof(prompt), "Enter the number of rows of
%s: ", name);

    if (!read_long_in_range(prompt, 1, MAX_MATRIX_DIM, out_rows))
{
        return false;
    }

    snprintf(
        prompt, sizeof(prompt), "Enter the number of columns of
%s: ", name
    );

    return read_long_in_range(prompt, 1, MAX_MATRIX_DIM,
out_cols);
}

bool read_matrix(
    const char *const name,
    Matrix matrix,
    const long rows,
    const long cols
) {
    char prompt[PROMPT_SIZE];

    snprintf(
        prompt, sizeof(prompt), "Fill %s with random numbers?
(Y/n): ", name
```

```
    );

    const bool random_fill = confirm(prompt, true);

    if (feof(stdin)) {
        return false;
    }

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            if (random_fill) {
                const int r = rand(); // NOLINT(*-msc50-cpp)
                matrix[i][j] = r % (2 * RANDOM_ABS + 1) -
RANDOM_ABS;
                continue;
            }

            snprintf(prompt, sizeof(prompt), "%s[%ld][%ld] = ",
name, i, j);

            if (!read_long_in_range(
                    prompt, -MAX_ELEMENT_ABS, MAX_ELEMENT_ABS,
&matrix[i][j]
                )) {
                return false;
            }
        }
    }

    return true;
}

void print_matrix(
    const char *const name,
    Matrix matrix,
    const long rows,
    const long cols
) {
    char text[32];
    size_t width = 1;

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            snprintf(text, sizeof(text), "%ld", matrix[i][j]);

            if (strlen(text) > width) {
                width = strlen(text);
```

```
            }
        }
    }

    printf("%s (%ld x %ld) =\n", name, rows, cols);

    for (long i = 0; i < rows; ++i) {
        for (long j = 0; j < cols; ++j) {
            printf(j == 0 ? "%*ld" : " %*ld", (int)width,
matrix[i][j]);
        }

        puts("");
    }
}
```

APPENDIX H

Source code of the “utils.c” file

```
#include "utils.h"

#include <ctype.h>
#include <errno.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { MAX_READ_VALUES = 8 };

typedef bool (*token_parser_fn)(const char *str, char **endptr,
void *out);

static bool is_buffer_empty(const char *ptr) {
    while (*ptr != '\0') {
        if (!isspace((unsigned char)*ptr)) {
            return false;
        }

        ptr++;
    }

    return true;
}

static bool parse_double_token(const char *str, char **endptr,
void *out) {
    errno = 0;
    *(double *)out = strtod(str, endptr);

    return *endptr != str && errno == 0;
}

static bool parse_long_token(const char *str, char **endptr, void
*out) {
    errno = 0;
    *(long *)out = strtol(str, endptr, 10);

    return *endptr != str && errno == 0;
}

static bool read_line_values(
    const char *prompt,
```

```
    const size_t count,
    void *const outs[],
    const token_parser_fn parse
) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    if (prompt != NULL) {
        printf("%s", prompt);
    }

    char buffer[256];
    if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
        return false;
    }

    buffer[strcspn(buffer, "\r\n")] = '\0';

    const char *ptr = buffer;

    for (size_t i = 0; i < count; ++i) {
        if (outs[i] == NULL) {
            return false;
        }

        char *endptr;
        if (!parse(ptr, &endptr, outs[i])) {
            return false;
        }

        ptr = endptr;
    }

    return is_buffer_empty(ptr);
}

bool read_double(const char *prompt, double *out_value) {
    return read_doubles(prompt, 1, out_value);
}

bool read_doubles(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];
```

```
    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, double *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs,
parse_double_token);
}

bool read_long(const char *prompt, long *out_value) {
    return read_longs(prompt, 1, out_value);
}

bool read_longs(const char *prompt, const size_t count, ...) {
    if (count == 0 || count > MAX_READ_VALUES) {
        return false;
    }

    void *outs[MAX_READ_VALUES];

    va_list args;
    va_start(args, count);

    for (size_t i = 0; i < count; ++i) {
        outs[i] = va_arg(args, long *);
    }

    va_end(args);

    return read_line_values(prompt, count, outs,
parse_long_token);
}

bool read_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value
) {
    while (true) {
        if (read_long(prompt, out_value)) {
            if (*out_value >= min && *out_value <= max) {
```

```
                return true;
            }

            printf("Error: value must be between %ld and %ld.\n",
min, max);
        } else if (feof(stdin)) {
            return false;
        } else {
            puts("Invalid input. Please enter an integer.");
        }
    }
}

bool read_optional_long_in_range(
    const char *prompt,
    const long min,
    const long max,
    long *out_value,
    bool *out_provided
) {
    char buffer[256];

    while (true) {
        printf("%s", prompt);

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        if (is_buffer_empty(buffer)) {
            *out_provided = false;
            return true;
        }

        char *endptr;
        long value;

        if (!parse_long_token(buffer, &endptr, &value) ||
            !is_buffer_empty(endptr)) {
            puts("Invalid input. Please enter an integer or press
Enter.");
        } else if (value < min || value > max) {
            printf("Error: value must be between %ld and %ld.\n",
min, max);
        } else {
            *out_value = value;
            *out_provided = true;
```

```
            return true;
        }
    }
}

static bool equals_ignore_case(
    const char *str,
    const size_t len,
    const char *word
) {
    if (strlen(word) != len) {
        return false;
    }

    for (size_t i = 0; i < len; ++i) {
        if (tolower((unsigned char)str[i]) != word[i]) {
            return false;
        }
    }

    return true;
}

bool confirm(const char *prompt, const bool default_answer) {
    char buffer[16];

    while (true) {
        if (prompt != NULL) {
            printf("%s", prompt);
        }

        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            return false;
        }

        const bool overflow = strchr(buffer, '\n') == NULL &&
!feof(stdin);
        if (overflow) {
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF)
                ;
        } else {
            const char *start = buffer;
            while (isspace((unsigned char)*start)) {
                start++;
            }
```

```
            size_t len = strlen(start);
            while (len > 0 && isspace((unsigned char)start[len -
1])) {
                len--;
            }

            if (len == 0) {
                return default_answer;
            }

            if (equals_ignore_case(start, len, "y") ||
                equals_ignore_case(start, len, "yes")) {
                return true;
            }

            if (equals_ignore_case(start, len, "n") ||
                equals_ignore_case(start, len, "no")) {
                return false;
            }
        }

        printf("Please answer 'y' or 'n'.\n");
    }
}
```